

ROS 2: control and simulation

Dawid Seredyński

Warsaw University of Technology

June 2026

Aim of this lecture

To understand *ros2_control*:

- motivation
- concept
- structure
- behavior
- design patterns

To understand *Gazebo Sim*:

- design
- modeling
- simulation time
- *ROS 2* integration
- *ros2_control* integration

Learning goals

By the end of this module, you should be able to:

- build and configure a *ros2_control controller*
- run a *Gazebo Sim* simulator and integrate it with *ROS 2*
- configure a *controller* inside *Gazebo Sim*

High-level *ROS 2*

Introduction

Control

ros2_control

Examples

Controller manager

Switching

Failures

Design

Chainable controllers

Gazebo Sim

Alternatives

Architecture

SDF

Simulation

ROS 2 integration

ros2_control

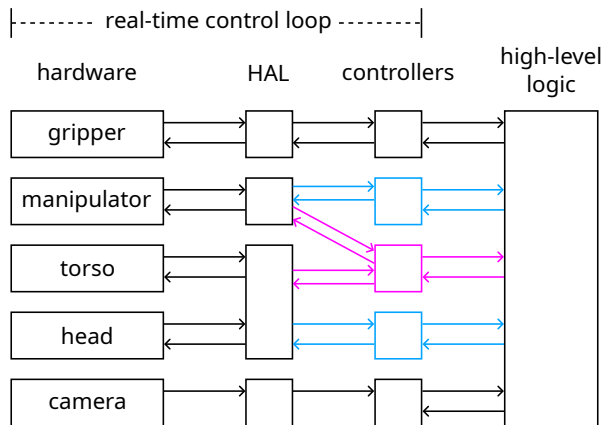
integration

Tools and resources

ROS 2 allows to integrate:

- simulation: e.g. *Gazebo Sim*
- control: *ros2_control*
- planning: *Movel*

Motivation for *ros2_control*



ros2_control

A `controller_manager`:

- is a *node*
- manages:
 - ◊ *hardware components*
 - ◊ *controllers*

Every `hardware component` and every `controller` is

- a *plugin* (managed by *pluginlib*)
- *lifecycle-managed*

Configuration locations:

- *hardware component* – in the *URDF*, using `<ros2_control>` tags
- *controller* – ROS 2 *parameters*, usually in `ros2_controllers.yaml`

A *ros2_control* **interface** is a one-way communication channel:

- between a *controller* and a *hardware component* [or a *chainable controller*]
- with an unique name
- with primitive, numerical data types

Types of *hardware interfaces* (i.e. *interfaces* of a *hardware component*):

- *command_interface*
- *state_interface*

A *hardware component* provides *interfaces*

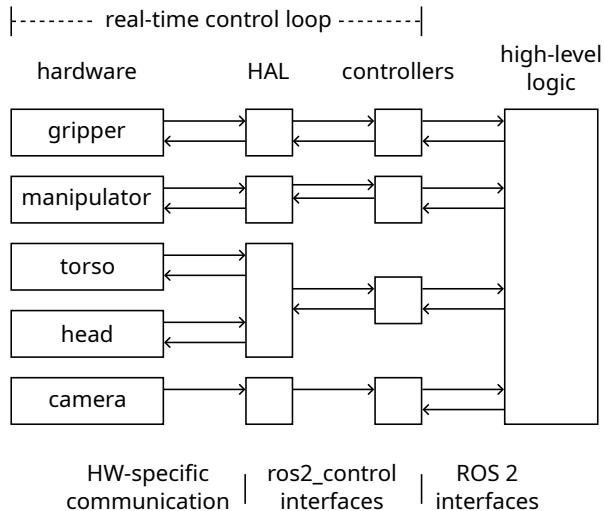
A *controller* requires *interfaces*

Connections are determined by:

- a set of active *controllers* and *hardware interfaces*
- matching names of provided *interfaces* and required *interfaces*

Note: the term “*interface*” overlap in *ros2_control* and *ROS 2*

ros2_control interfaces and ROS 2 interfaces



Example: configuration of a *hardware component*

Configuration of a *hardware component* is in URDF. It defines:

- type of ROS 2 plugin to communicate with HW
- interfaces the hardware provides

```
<ros2_control name="HeadHw" type="system">
  <hardware>
    <plugin>my_robot/HeadHw</plugin>
  </hardware>

  <joint name="head_pan">
    <command_interface name="position"/>
    <state_interface name="position"/>
    <state_interface name="velocity"/>
  </joint>

  <joint name="head_tilt">
    <command_interface name="position"/>
    <state_interface name="position"/>
    <state_interface name="velocity"/>
  </joint>
</ros2_control>
```

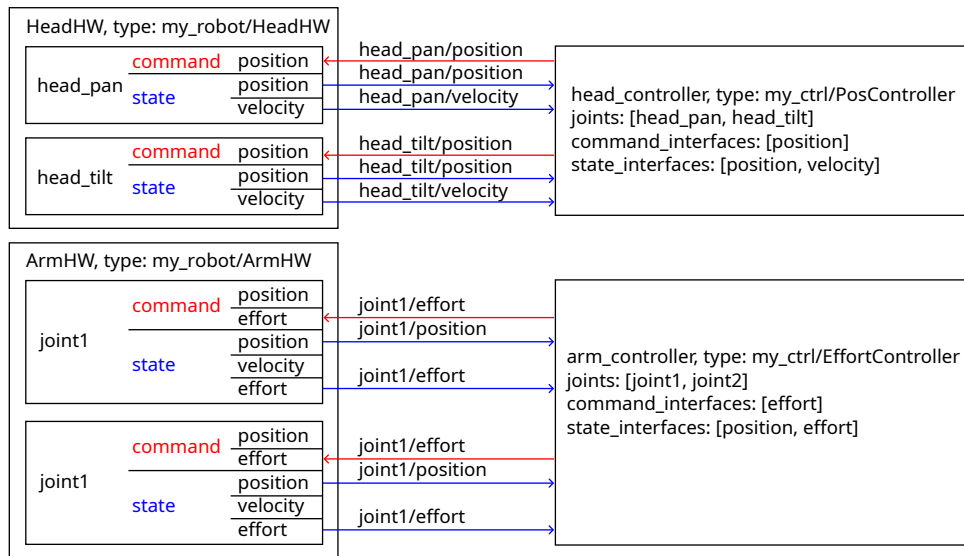
Example: configuration of *controllers*

```
controller_manager:
  ros__parameters:
    update_rate: 1000
    head_controller:
      type: my_ctrl/PosController
    arm_controller:
      type: my_ctrl/EffortController

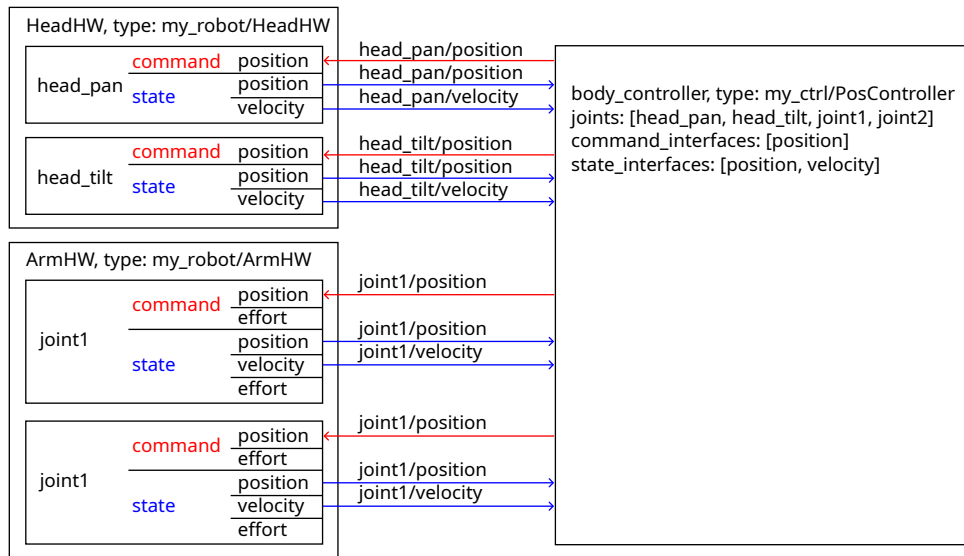
head_controller:
  ros__parameters:
    joints: [head_pan, head_tilt]
    command_interfaces: [position]
    state_interfaces: [position, velocity]

arm_controller:
  ros__parameters:
    joints: [joint1, joint2]
    command_interfaces: [effort]
    state_interfaces: [position, effort]
```

Example: claiming interfaces



Example: claiming interfaces



A *node* for *controller_manager*

A *controller_manager* can be run:

- as a separate process (pkg://controller_manager/ros2_control_node)
 - ◊ e.g. in a *launch file*
- as a class instance in a process
 - ◊ e.g. a *gz-plugin* running in *Gazebo Sim*

Note: *gz-plugin* is a plugin for *Gazebo Sim* (not related to *pluginlib*)

Loading of *ros2_control*

A *controller_manager* starts, and:

- reads ROS 2 *parameters*, and:
 - ◊ loads configuration of *controllers*
 - ◊ stores *interfaces* of *controllers*
- reads *robot_description*, and:
 - ◊ parses `<ros2_control>` tags
 - ◊ loads *plugins* for *hardware components*
 - ◊ stores available *hardware interfaces* in *resource_manager*
 - ◊ activates (by default) *hardware components*
 - initial states of *hardware components* can be specified in *parameters* of *controller_manager*

At this stage:

- *hardware components* are loaded, and may be initialized, configured and activated
- *controllers* are not loaded

Spawning *controllers*

spawner★ – a *node* that loads, configures and start controllers
(pkg://controller_manager/spawner)

- calls *service* controller_manager/load_controller
- multiple controllers can be spawned at once

In result:

- *controller_manager* uses *pluginlib* to load the *controller* class
- *controller* instance is created
- *controller* can be configured and/or activated on load or later

spawner requires only names of *controllers*

- *plugin* type of each *controllers* and its configuration is in the *parameters*

Behaviors of *spawner*:

- load only ($\rightarrow$ *unconfigured*):
`ros2 run controller_manager spawner my_controller -load-only`
- load and configure ($\rightarrow$ *inactive*):
`ros2 run controller_manager spawner my_controller -inactive`
- default ($\rightarrow$ *active*):
`ros2 run controller_manager spawner my_controller`

unspawner – stops and unloads controllers

- calls *service* `controller_manager/unload_controller`
- required e.g. to reload a *controller* after build

hardware_spawner - activates and configures a hardware component

- if not configured / activated on load

How to activate / switch *controllers*?

Motivation: to change structure of *controllers*, behavior, control law

Ask the *controller_manager* to change *lifecycle* states of *controllers*

This can be done by:

- *spawner*
- *CLI*: `ros2 control switch_controllers`
- *controller_manager* services directly
- a GUI *node*: `pkg://rqt_controller_manager/rqt_controller_manager`

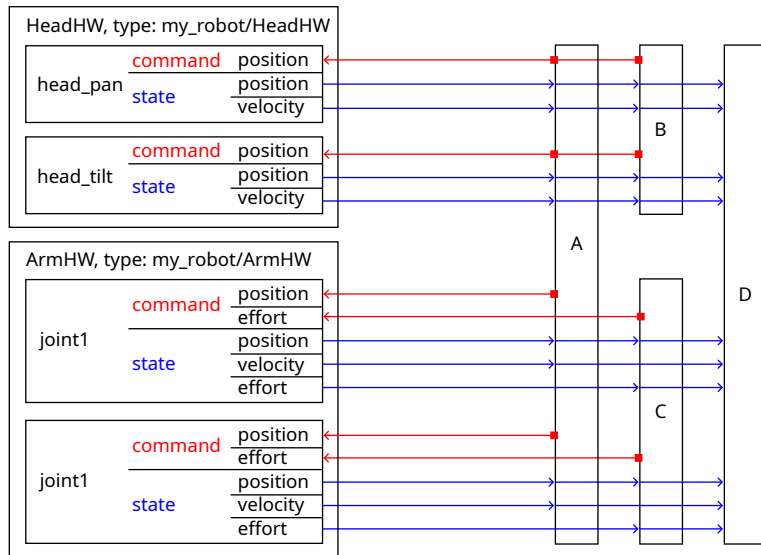
Parameters for switch_controllers:

- a list of *controllers* to deactivate
- a list of *controllers* to activate
- mode:
 - ◊ STRICT
 - controller switching is all-or-nothing
 - if any requested activation/deactivation fails, the switch is aborted (no effect)
 - default mode
 - ◊ or BEST_EFFORT
 - partial switching may occur
- timeout
- name of *controller_manager*

In result:

- multiple *controllers* can be switched (activated or deactivated)
- switch happens in one step of the control loop
- it is possible to restart active *controllers*

Example: conflicts



Switch failures

- *controller* name is wrong or is not loaded
- *controller* is in the wrong lifecycle state
- *controller* callbacks `on_activate()` or `on_deactivate()` failed
- required *hardware interface* does not exist
- *command_interface* already claimed – another active controller already owns it
- required *hardware interface* is not available – hardware component is inactive/unconfigured/failed, so its interfaces cannot be claimed
- *hardware component* rejects command-mode switch, e.g. hardware does not allow switching from position mode to velocity mode
- chained-controller dependency problem – a controller depends on reference interfaces from another controller, but those interfaces are not available
- switch timeout – *hardware interface* does not become ready before the requested timeout
- controller throws an exception – if `handle_exceptions` is enabled

Failure of a *controller*

An active *controller* may be deactivated when:

- its `update()` failed (returned `ERROR`)
- it is chained with a *controller* whose `update()` failed
- it uses any *hardware interface* of a *hardware component* that failed to `read()` or `write()`

In result, a *controller_manager* tries to switch to *fallback controllers*

fallback controllers:

- are specified in *parameter* `fallback_controllers` of a *controller*
- they are activated using `STRICT` mode
- it is recommended to carefully design fallback and test it thoroughly in simulation

Design of a *hardware component*

A *hardware component* is initialized from the *URDF* (*interfaces*, joints, sensors, *parameters*)

Some methods may require implementation:

- from `rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface`★
 - ◊ optional (virtual): `on_configure`, `on_cleanup`, `on_shutdown`, `on_activate`, `on_deactivate`, `on_error`, destructor
- from `hardware_interface::HardwareComponentInterface`★
 - ◊ obligatory (pure virtual): `read`
 - ◊ optional, frequently reimplemented: `write`, `on_init`
 - ◊ optional, other: exporting interfaces, command mode switch, etc.
- from `SystemInterface`★ or `ActuatorInterface`★: `write` – obligatory
- from `SensorInterface`★: `write` – not available (final)

Design of a *controller*

A *controller* is initialized from *parameters* (*interfaces*, joints, specific *parameters*)

Some methods may require implementation:

- from `rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface`★
 - ◊ optional (virtual): `on_configure`, `on_cleanup`, `on_shutdown`, `on_activate`, `on_deactivate`, `on_error`, `destructor`
- from `controller_interface::ControllerInterfaceBase`★
 - ◊ obligatory: `on_init`, `update`, `command_interface_configuration`, `state_interface_configuration`
 - ◊ optional: `assign_interfaces`, `release_interfaces`, `define_custom_node_options`
 - ◊ implemented (final) in derived classes: `ControllerInterface`★ and `ChainableControllerInterface`★:
`is_chainable`, `is_in_chained_mode`, `set_chained_mode`, `export_state_interfaces`, `export_reference_interfaces`

Multiple robots

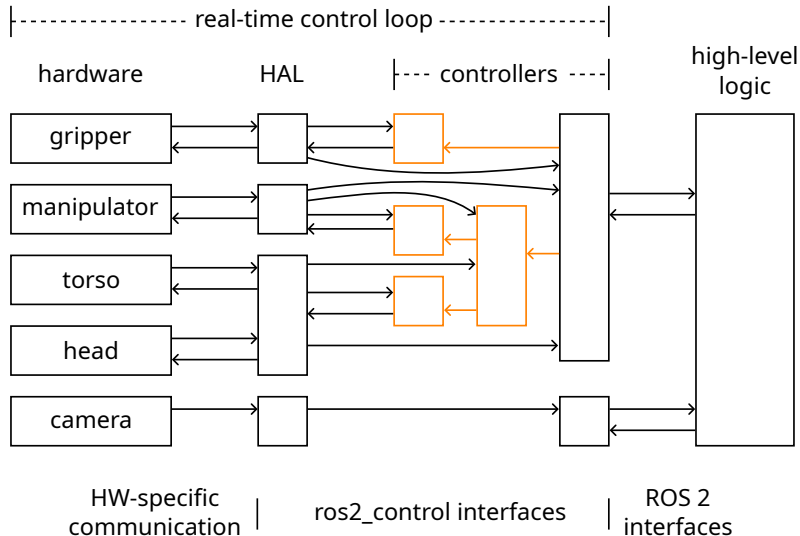
Multiple *controller_managers* can be running in one system

- e.g. for multiple robots
- each *controller_manager node* is identified by its name
- each robot should have its own namespace

ros2 control <command>

- cleanup_controller
- list_controller_types
- list_controllers
- list_hardware_components
- list_hardware_interfaces
- load_controller
- reload_controller_libraries
- set_controller_state
- set_hardware_component_state
- switch_controllers
- unload_controller
- view_controller_chains

Structure of *chainable controllers*



Concept of a *chainable controller*

A *chainable controller*★ has additional interfaces to a higher-level *controller*

- it enables cascade control
- it can operate either in chained or non-chained mode

A non-chainable *controller* has:

- required command interfaces
- required state interfaces

A *chainable controller* additionally has:

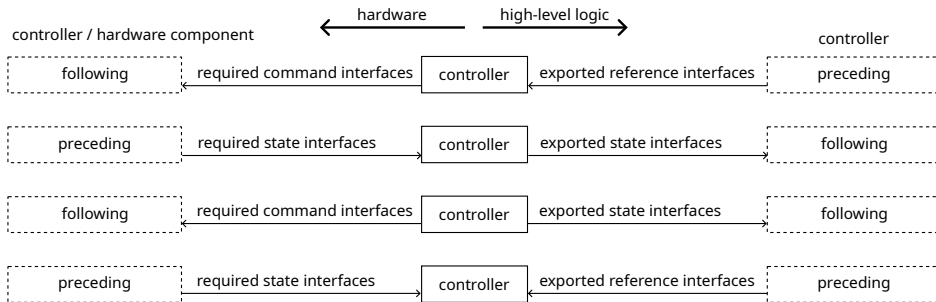
- exported reference interfaces
- exported state interfaces

Design of a *chainable controller*

A *chainable controller* should implement the same methods as a regular *controller* and additionally:

- from `controller_interface::ChainableControllerInterface`★
 - ◇ obligatory: `update_reference_from_subscribers`,
`update_and_write_commands`
 - ◇ optional: `on_export_state_interfaces`,
`on_export_state_interfaces_list`,
`on_export_reference_interfaces`,
`on_export_reference_interfaces_list`, `on_set_chained_mode`

Order



- update order
 - ◊ all preceding first
- individual activation / deactivation order★
 - ◊ all following first / all preceding first

Debugging structure of *controllers*

`ros2 control view_controller_chains`

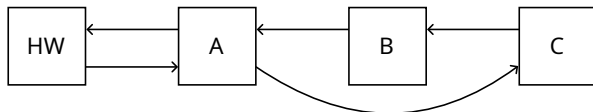
– is broken, cannot parse *interface* names properly

controller_manager prints the loading procedure and *controller* chains when in the highest debug level

To run *controller_manager* with the highest debug level:

- for a stand-alone *node* `pkg://controller_manager/ros2_control_node`, define additional arguments:
`--ros-args -log-level controller_manager:=debug`
- for *controller_manager* running inside *Gazebo Sim* – call a *service* before *controllers* are spawned:
`ros2 service call /controller_manager/set_logger_levels ...`

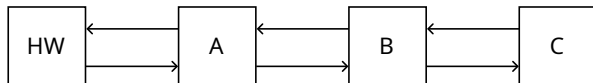
Controllers with a loop (bad design)



spawn order	controller order
A, B, C	C, B, A
B, A, C	C, B, A
A, C, B	A, C, B
B, C, A	A, C, B
C, B, A	A, C, B
C, A, B	B, A, C

Cycles required for HW $\rightarrow$ controllers $\rightarrow$ HW propagation: 2

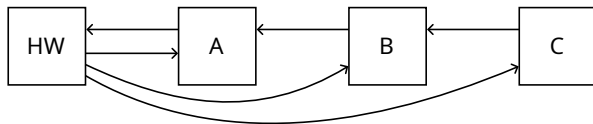
Controllers with a loop (bad design)



spawn order	controller order
A, B, C	A, B, C
B, C, A	A, B, C
B, A, C	A, B, C
C, B, A	A, B, C
A, C, B	B, A, C
C, A, B	B, A, C

Cycles required for HW → controllers → HW propagation: 3

Controllers without loops (ok)



spawn order	controller order
A, B, C	C, B, A
A, C, B	C, B, A
B, C, A	C, B, A
B, A, C	C, B, A
C, A, B	C, B, A
C, B, A	C, B, A

Cycles required for HW → controllers → HW propagation: 1

The Good, the Bad and the Ugly

Good: avoid loops in chained controllers

- if a loop is needed – you should redesign the controller structure

Bad: use a specific spawn order to get a desired order of updates

- not documented, may change in future releases

Ugly:

- use `realtime_tools::RealtimePublisher` on one side, and subscription with a realtime buffer on the other
 - ◊ it's asynchronous, may take random time to deliver
- or implement your own RT-safe, intra process communication

Simulators available for *ROS 2*

simulator	properties
<i>Gazebo Sim</i>	general robotics (mobile, manipulators), Nav2, <i>ros2_control</i> , sensors
Webots	education, prototyping, mobile robots, simple manipulators
MVSim	wheeled platforms, 2D / 2.5D, SLAM, Nav2
MuJoCo	contact dynamics, manipulation, RL, biomechanics, robot-oriented
Isaac Sim	photorealistic, synthetic data, AI, robot learning, digital twin

There are more simulators

- harder to integrate
- specialized

Gazebo Sim★ – an open-source robotics simulator

- **dynamics simulation**: multiple high-performance physics engines
- **advanced 3D graphics**: rendering engines such as OGRE v2 for realistic rendering of environments with high-quality lighting, shadows, and textures
- **sensors and noise models**: sensor data, optionally with noise, from laser range finders, 2D/3D cameras, Kinect style sensors, contact sensors, force-torque, IMU, GPS, and more
- **gz-plugins**: custom plugins for robot, sensor, and environment control
- **GUI**: plugin-based graphical interfaces
- **simulation models**: numerous robots available through Gazebo Fuel
- **gz-transport**: message passing using **gz-topic** and **gz-service** (different than in ROS 2)
- **TCP/IP Transport**: simulation on remote servers; socket-based *gz-transport*
- **command line tools**: for increased simulation introspection and control

Versions and compatibility

ROS 2	Gazebo			
	Fortress (LTS)	Harmonic (LTS)	Ionic	Jetty (LTS)
Rolling	×	!	!	✓
Lyrical (LTS)	×	!	!	✓
Kilted	×	!	✓	×
Jazzy (LTS)	×	✓	×	×
Humble (LTS)	✓	!	×	×

Architecture of *Gazebo Sim*

Backend: `gz-server` process

Frontend: `gz-client` process

Core functionality:

gz-client – *gz-server* communication + various *gz-plugins*, e.g.:

- physics, rendering, sensors, controllers (*Gazebo Sim*, not *ROS 2*), GUI

Software design pattern: **Entity Component System (ECS)**

- entity – anything in the simulation scene (a model, link, light, actor, etc.)
- components – entity's characteristics (pose, name, geometry, etc.).

Recommended version for *Ubuntu 24.04 Noble* + *ROS 2 Jazzy*:
Gazebo Harmonic (8.x series)

Note: some names overlap in *Gazebo Sim* and *ROS 2*

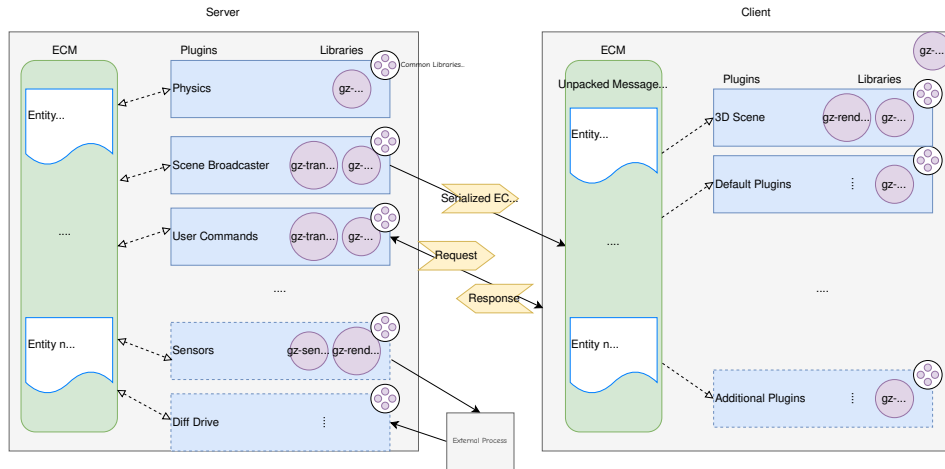


image source: <https://gazebo.org/docs/latest/architecture>

Introduction

Control

ros2_control
Examples
Controller manager
Switching
Failures
Design
Chainable controllers

Gazebo Sim

Alternatives
Architecture
SDF
Simulation
ROS 2 integration
ros2_control integration
Tools and resources

Simulation Description Format (SDF)★ – description of simulated world
(robots + environment)

SDF is broader than *URDF*

A “robot” in *URDF* → a “model” in *SDF*

Aspect	URDF	SDF
ecosystem	<i>ROS 2</i>	<i>Gazebo Sim</i>
scope	usually one robot	a whole simulation world
describes	a robot model: links, joints, geometry, inertial data, collision shapes, etc.	a simulation entity or world: robots, objects, environments, physics, lights, terrain, sensors, plugins, etc.
kinematic structure	tree of links and joints; kinematic loops unsupported	graph-style model structure; can represent closed loops
simulation detail	limited core simulation semantics; <gazebo> extension tags★	physics engines, surface, friction, contacts, static/dynamic objects
pose, frames	<origin> tied to links / joints	<pose> with frame semantics
composition	<i>xacro</i> to generate reusable URDF	supports includes and nested models directly

SDF is broader than *URDF*

- typical conversion *URDF* $\rightarrow$ *SDF*

Gazebo Sim can load a *URDF* at startup

- *URDF* is converted to *SDF*
- conversion generates lots of warnings

Better approach:

- maintain a *URDF* / *xacro* for ROS 2 (*robot_description*, *rviz2*, *ros2_control*)
- generate a *SDF* from *URDF* / *xacro* and keep the *SDF* up to date
- use model spawning in *Gazebo Sim*
 - ◇ load a simulated world (*SDF*) without the robot
 - ◇ spawn a robot (*SDF*) by using a *gz-service*

Simulation time★

Two clocks:

- *wall time* (or real time): actual time on your computer
- *simulation time*: time inside the simulated world

Different starting point, different rate – example:

- *wall time*: 1779356077.28 (over 56 years), elapsed: 287.51
- *simulation time*: 230.12, elapsed: 230.12

Real-time factor (RTF):

$$RTF = \frac{\text{simulation_time_elapsed}}{\text{wall_time_elapsed}} \quad (\text{in some time window})$$

Simulation time control★:

- available in *gz-client* (GUI) or through *gz-service*
- controls: stop, run, step, reset
- displays: *RTF*, *simulation time*, *wall time*, iterations

Important simulation parameters★

- `max_step_size` (default: 0.001 s)
- `real_time_factor` (1.0), or `real_time_update_rate` (1000 Hz)

$$\text{real_time_factor} = \text{real_time_update_rate} \cdot \text{max_step_size}$$

- `max_contacts` (20)
- physics engine (*DART*)
 - ◇ solver type (dantzig)
 - ◇ collision detector (ode)
- sensor parameters
 - ◇ `update_rate`
 - ◇ resolution / number of samples

Initialization and the main loop

1. Configure

- ◇ one-time setup of the simulated world, reading *SDF* parameters, creating components

2. The main loop:

- ◇ PreUpdate
 - write desired commands before physics, e.g. forces, joint commands, velocity commands
- ◇ Update
 - main physics/simulation step
- ◇ PostUpdate
 - read-only phase for outputs, sensors, odometry, state, logging

Each time a new model is spawned, its *gz-plugins* are loaded and configured

Introduction

Control

ros2_control

Examples

Controller manager

Switching

Failures

Design

Chainable controllers

Gazebo Sim

Alternatives

Architecture

SDF

Simulation

ROS 2 integration

ros2_control

integration

Tools and resources

ROS 2 + Gazebo Sim integration★

gz-transport is different than *ROS 2 interfaces*

`ros_gz_bridge` – provides a network bridge *ROS 2* $\longleftrightarrow$ *Gazebo Sim*

- enables the exchange of messages between *ROS 2* and *Gazebo Transport*
- limited to only certain message types★
- few *gz-services* can be bridged★

```
ros2 run ros_gz_bridge parameter_bridge <topic>@<tp_ros>@<tp_gz>
```

- the second delimiter:
 - ◇ @ – for bi-directional
 - ◇ [– *Gazebo Sim* $\rightarrow$ *ROS 2*
 - ◇] – *ROS 2* $\rightarrow$ *Gazebo Sim*

```
pkg://ros_gz_bridge/launch/ros_gz_bridge.launch.py
```

- bridge parameters loaded from `.yaml`

Launching *Gazebo Sim* in *ROS 2*★

`ros_gz_sim` – a package that provides *launch files* for:

- running *Gazebo Sim* from a *launch file*
 - ◇ *gz-server* only
 - ◇ *gz-server* and *gz-client*
- spawning a model
- removing a model

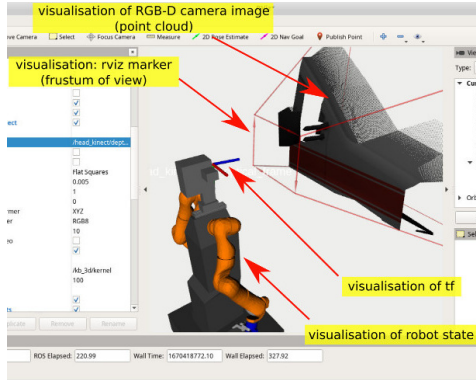
Loading a robot into simulation – two options:

- a world file (*SDF*) with a robot model (*SDF*) included – at startup
- spawning a robot model into a simulated world

Simulation time in *ROS 2*

- clock bridge is required: `/clock` topic *Gazebo Sim* → *ROS 2*
- set `use_sim_time:=true` *parameter* for all *nodes* that use *simulation time*

gz-client, rviz2



ROS 2: control
and simulation

Dawid
Seredyński

Introduction

Control

`ros2_control`

Examples

Controller manager

Switching

Failures

Design

Chainable controllers

Gazebo Sim

Alternatives

Architecture

SDF

Simulation

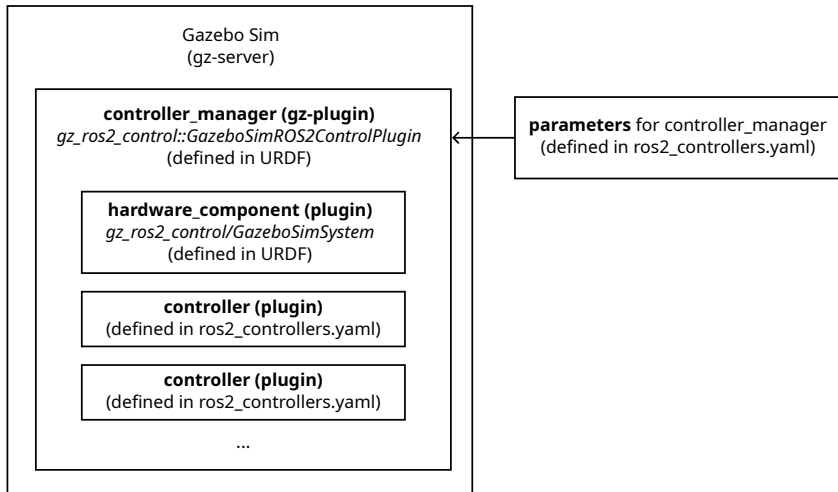
ROS 2 integration

`ros2_control`

integration

Tools and resources

ros2_control integration



ros2_control integration (URDF)

Load *controller_manager* as a *gz-plugin*

```
<gazebo>
  <plugin filename="libgz_ros2_control-system.so"
          name="gz_ros2_control::GazeboSimROS2ControlPlugin">
    <parameters>.../ros2_controllers.yaml</parameters>
  </plugin>
</gazebo>
```

Load *hardware component* for *Gazebo Sim* in *controller_manager* as a *ros2_control plugin*

```
<ros2_control name="${name}" type="system">
  <hardware>
    <plugin>gz_ros2_control/GazeboSimSystem</plugin>
  </hardware>
  <joint name="torso_0_joint">
    <command_interface name="position"/>
    ...
```

Tools and resources★

Command line tools:

- `gz sim` – starts *Gazebo Sim* (*gz-server*, *gz-client*, headless, paused, auto-running)
- `gz topic` - inspect, echo, and publish *gz-transport* *gz-topics*
- `gz service` – list and call *gz-transport* *gz-services*, e.g. world control, pause, run, reset, entity creation commands
- `gz model` – query model information from a running simulation
- `gz fuel` – list, download, upload, and manage online simulation assets
- `gz launch` – runs *gz-launch* descriptions for *Gazebo Sim* processes and plugins★
- `gz sdf` – *SDF* inspection and conversion

***Gazebo Sim fuel*★** – server with models (3300) and worlds (250)

Introduction

Control

`ros2_control`

Examples

Controller manager

Switching

Failures

Design

Chainable controllers

Gazebo Sim

Alternatives

Architecture

SDF

Simulation

ROS 2 integration

`ros2_control`

integration

Tools and resources

Questions?